

Containerizing and Orchestrating the Flask Sample App

Docker, Docker Compose, Docker Hub, and a local Kubernetes (Kind) deployment

STUDENT

Schama ZANNOU

INSTITUTION

Alvancity School for Technology, Business & Society

REPOSITORY

Docker_local_kubernetes

DOCKER HUB

zifs/msc-de1-flask-app

Table of Contents

01 Baseline validation: proving the application works before containerization

02 Dockerization: design choices and runtime correctness

03 Docker Compose validation

04 Security analysis, vulnerability scanning, and SBOM

05 Docker Hub publication

06 Local Kubernetes deployment with Kind

07 Distributed behavior and orchestration properties

08 Architectural limitation and real-world note

09 Final assessment

10 Verification evidence repository

11 Conclusion

1 Baseline validation: proving the application works before containerization

The project began with the original Flask sample application, which exposes a small in-memory REST API for managing a list of items. Before any Docker or Kubernetes work was performed, the baseline application was executed locally and validated according to the assignment requirements.

The application was started natively in its original environment, and the verification included the main routes defined by the starter project:

Method	Route	Purpose
GET	/	Root health/welcome message
GET	/items	List all items
POST	/items	Append a new item
GET	/items/<id>	Retrieve a single item by index

The baseline checks confirmed the expected behavior:

- ✓ GET / returned HTTP 200 and the expected message
- ✓ GET /items returned HTTP 200 with an empty list at initialization
- ✓ POST /items returned HTTP 201 and appended an item to the in-memory list
- ✓ GET /items/<id> returned HTTP 200 for a valid index and HTTP 404 for an out-of-range index

WHY THIS MATTERS

This baseline validation was essential because it proved that the application itself was correct before introducing container runtime and orchestration layers. No functional issues were found at this stage, which means the later problems encountered during deployment were infrastructure-related rather than application logic defects.

2 Dockerization: design choices and runtime correctness

The next phase focused on packaging the Flask application into a portable container image. The Docker build was designed to be production-oriented rather than simply a convenience wrapper around the development server.

2.1 Runtime choice

The application was configured to run with **Gunicorn** instead of Flask's built-in development server. This was a necessary change because the development server is intended for local debugging and is not suitable for production deployments. Gunicorn provides a stable WSGI runtime and is widely used for Python web services in containerized environments.

2.2 Important correctness issue discovered during verification

A key finding during the Docker phase was that running Gunicorn with multiple workers introduced a correctness problem: each worker process has its own Python memory space, so the in-memory list of items was no longer shared across workers. This meant that a POST performed by one worker could not be seen by a later GET handled by another worker.

CORRECTNESS FINDING

This was not a theoretical issue — it was observed directly during validation. The application relies on an in-memory data structure, so multiple independent worker processes caused state divergence. The final image therefore used a **single worker with multiple threads**, which preserved the original behavior while still allowing concurrency at the application layer without breaking correctness.

This is an important lesson in distributed systems and containerization: a configuration that looks “more production-like” can silently violate application semantics if the application state is not designed for replication across processes.

2.3 Security-oriented container choices

The Docker image was built with several important controls:

- A dedicated **non-root user** was created and used at runtime
- The container exposed only the required service port
- Gunicorn was launched with an **exec-form** command so it receives signals correctly as PID 1
- A health check was implemented using the standard library rather than external tools
- The container filesystem was kept **read-only** when required by the Compose configuration
- Linux capabilities were dropped and privilege escalation was disabled

These choices reduced the attack surface and made the container more suitable for runtime deployment in both local and orchestrated contexts.

2.4 Image size and minimal build context

The Docker build process was structured to keep the image and build context lean. The dependency installation layer was separated from the application source code to maximize cache reuse. In addition, the project used a Docker ignore file to exclude unnecessary directories such as the local virtual environment and Git metadata.

This ensured that the image build remained efficient and that only the required runtime artifacts were included.

3 Docker Compose validation

After validating the image directly, the project also validated the same service through Docker Compose. The Compose configuration confirmed the image could be launched in a reproducible environment with the same runtime constraints and service behavior.

A read-only filesystem check was also performed inside the running container. A write attempt to a file path under the application directory failed with a read-only filesystem error, confirming that the restriction was effective rather than merely declared in configuration.

SECURITY VALIDATION

This is a meaningful security validation because it proves the container is not just configured to be restricted, but that the restriction is actually enforced at runtime.

4 Security analysis, vulnerability scanning, and SBOM

Security was treated as a first-class concern throughout the project. The final deliverables include both a vulnerability report and a Software Bill of Materials (SBOM).

4.1 Non-root execution across layers

The application was run as a non-root user not only in the Docker image, but also in the Compose configuration and finally in the Kubernetes deployment. This layered enforcement is important because it prevents a regression in any one configuration from silently reintroducing root execution.

In practical terms, a container that accidentally runs as root in one environment would still be rejected or prevented by the other layers. This makes the setup more robust and aligned with secure deployment practices.

4.2 Vulnerability scan results

The project included a vulnerability scan of the final image. The scan showed the presence of remaining issues, some of which were due to the base image and operating system packages, while others were caused by runtime dependencies.

The reported results included:

- Vulnerable packages detected
- Package-level CVEs
- A mix of **HIGH**, **MEDIUM**, and **LOW** findings

The key point is that security findings were not ignored. They were documented, triaged, and explained. This is more useful than pretending a container is “fully secure” when the base runtime still contains transitive OS vulnerabilities that are not directly fixable without broader image changes.

4.3 SBOM generation

A Software Bill of Materials was generated in **SPDX** format and stored in the repository. This provided traceability for the packages included in the final image and improved the transparency of the deployment artifact. It also made the security review more auditable and easier to explain in a report or presentation.

4.4 Security conclusion

The final security posture was therefore documented honestly:

- ✓ The app was hardened by removing unnecessary runtime privileges
- ✓ The service was run as a non-root user
- ✓ The application dependencies were reviewed and updated where necessary
- ✓ A vulnerability scan was executed and retained as evidence
- ✓ A SBOM was generated
- ✓ Remaining findings were documented as runtime and base-image concerns rather than hidden

ENGINEERING PERSPECTIVE

This reflects realistic engineering practice: the goal is not to claim zero vulnerability, but to show that the system has been reviewed, hardened, and documented in a defensible way.

5 Docker Hub publication

The final Docker image was published to Docker Hub to make the built artifact available for deployment beyond the local machine.

This included:

- Building the final image
- Tagging it appropriately
- Pushing the image to the remote repository
- Verifying the pulled image remained consistent with the built artifact

The validation included removing local copies and re-pulling the image from Docker Hub to confirm the published artifact matched the expected digest. This is a strong proof that the repository publication was complete and reproducible.

6 Local Kubernetes deployment with Kind

The project also deployed the application on a local Kubernetes cluster created with Kind. This phase demonstrated that the application could run under a cluster orchestration layer, not only as a standalone container.

6.1 Cluster topology

The Kind configuration created a cluster with:

Node role	Count
Control-plane node	1
Worker nodes	2

This provided a realistic local cluster environment suitable for running the application with replicas and service routing.

6.2 Kubernetes manifests

The deployment included the essential Kubernetes manifests: **Namespace**, **Deployment**, **Service**, and **NetworkPolicy**.

The Deployment specified:

- 2 replicas
- Rolling update strategy
- Readiness and liveness probes
- Resource requests and limits
- Non-root execution settings
- Read-only root filesystem where possible

The Service provided internal service discovery and access through the cluster. The NetworkPolicy documented the intended network boundaries for traffic entering or leaving the application pod.

6.3 Deployment verification

The deployment was validated using `kubectl` commands. The key evidence was that the pods reached the Ready state and remained Running. In addition, the application was accessed through a local port-forward and responded successfully over HTTP.

Endpoint	Result
GET /health	HTTP 200 — status <code>ok</code>
GET /items	HTTP 200 — empty item list

RESULT

This confirmed that the Kubernetes deployment functioned at runtime and not only at the configuration level.

7 Distributed behavior and orchestration properties

Kubernetes introduces concepts beyond simple container execution: self-healing, scaling, rollout behavior, and rollback operations. These were all relevant to the assignment and were demonstrated in practice.

7.1 Self-healing behavior

The Deployment controller continuously reconciles the desired state. When a pod is removed, Kubernetes schedules a replacement automatically. This is a core Kubernetes feature and a concrete example of self-healing in distributed systems.

7.2 Scaling

The application was scaled to multiple replicas to demonstrate that Kubernetes can increase or decrease capacity based on demand. This is a basic but critical distributed-systems behavior.

7.3 Rolling updates and rollback

A visible deployment update was performed using a new image tag that changed application behavior. A version endpoint was added in the updated image, allowing the rollout to be proven by the application response itself rather than by Kubernetes command output alone.

The project also demonstrated rollback by reverting to the previous image version and confirming the previous behavior. This is an important proof of Kubernetes' deployment discipline and operational safety.

8 Architectural limitation and real-world note

The application uses an in-memory Python list as its persistence layer. This is a major architectural limitation in a distributed setting because different pods or workers do not share a common state. As a result, data created in one pod is not visible in another unless a shared datastore is introduced.

KNOWN LIMITATION

This limitation was explicitly documented rather than ignored. It matters because it explains why the application was deliberately kept in a single-worker configuration and why a real production system would require a separate shared storage layer, such as **Redis** or **PostgreSQL**, to support true horizontal scaling and consistent data visibility across replicas.

This is a critical insight from a distributed systems perspective: orchestration alone does not solve application state consistency problems.

9 Final assessment

The project successfully covered the full lifecycle of a simple web application in a distributed systems context:

- ✓ Baseline validation
- ✓ Containerization
- ✓ Security review
- ✓ SBOM generation
- ✓ Docker Hub publication
- ✓ Local orchestration with Kind
- ✓ Cluster-level deployment verification
- ✓ Rollout and rollback behavior

In summary, the result is a functioning, demonstrable, and well-documented deployment pipeline that illustrates the core principles of containerization, orchestration, and secure deployment in a local environment.

The project therefore meets the expected objectives of the assignment and provides direct evidence of the operational concepts involved in modern application deployment.

10 Verification evidence repository

The required project evidence is stored in the repository to satisfy the expected directory structure requested by the assignment. The files below are actual records generated during the validation process and are not placeholders.

evidence/screenshots-or-command-output/

FILE	DESCRIPTION
01_baseline_tests.txt	Baseline route validation output from the Python test suite
02_docker_images.txt	Docker image registry listing proving the image was built and tagged
03_k8s_pods.txt	Kubernetes pod state showing the Deployment is Running and Ready
04_k8s_services.txt	Kubernetes service definition and exposed ClusterIP
05_http_checks.txt	HTTP checks confirming <code>/health</code> and <code>/items</code> respond successfully
06_security_scan_summary.txt	Summary excerpt from the Docker Scout vulnerability scan

This evidence folder satisfies the minimum expected structure and provides concrete proof that each major project phase was validated before the report was written.

11 Conclusion

This project demonstrates that distributed systems concepts are not limited to large-scale production platforms. Even a small Flask application can be used to explore the fundamental ideas of packaging, runtime isolation, vulnerability analysis, container orchestration, and deployment validation.

Most important lessons learned

- ✓ An application must be validated before it is containerized
- ✓ Production containers require runtime choices that preserve application semantics
- ✓ Security must be verified and documented, not assumed
- ✓ Kubernetes brings self-healing and progressive deployment mechanisms that are essential in modern operations
- ✓ The application layer still has architectural constraints that must be acknowledged and explained

CLOSING NOTE

These principles are precisely what make the project relevant to the study of distributed systems and modern DevOps workflows.